



8

18

2160 sp

نظري

كلية الهندسة المعلوماتية

السنة الثالثة

Advanced SQL

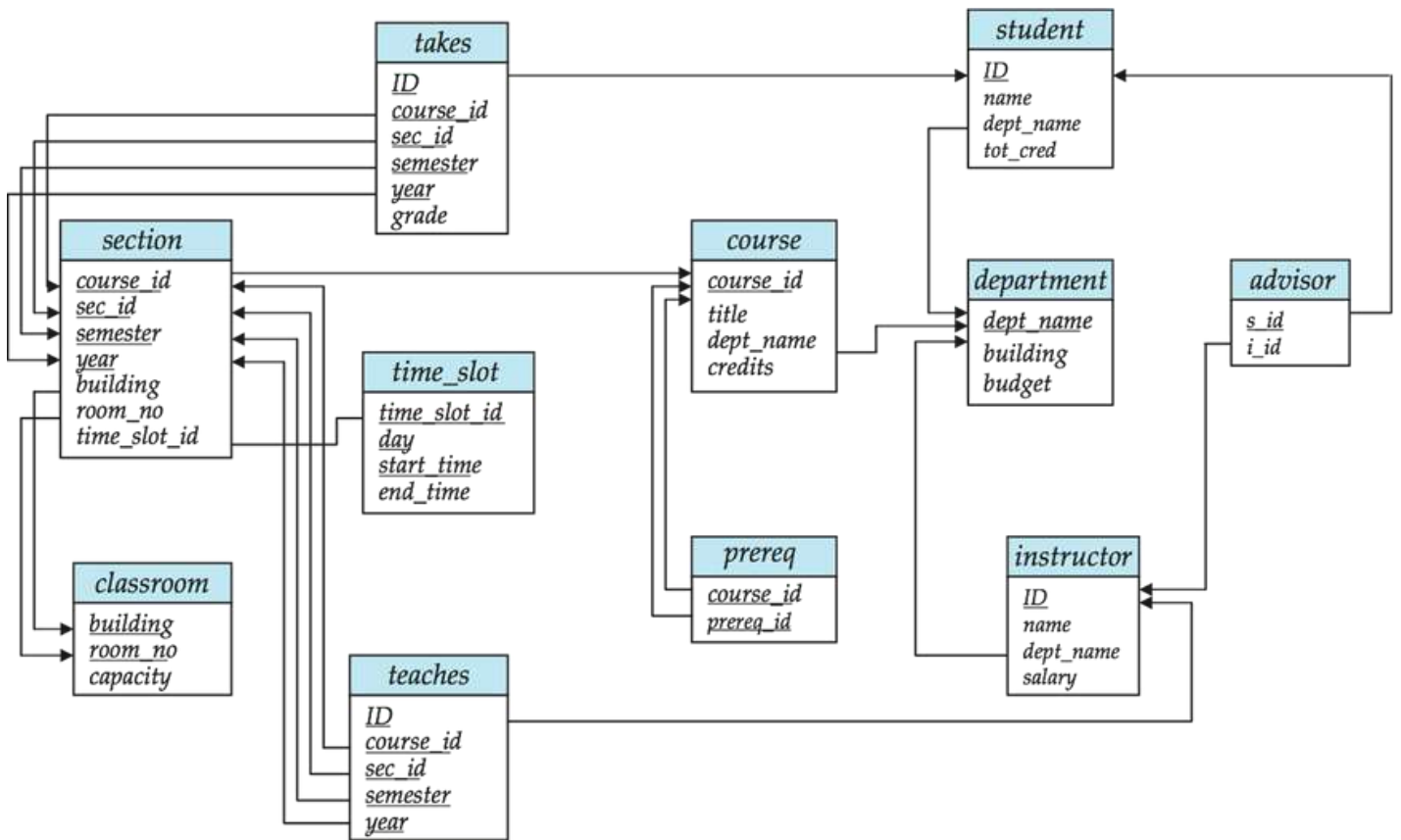
د. سعيد دسوقي



RB Informatics; 18/06/2026

قواعد معطيات

مخطط بنية معطيات جامعة:



اختبار العلاقات الفارغة (Testing for Empty Relations)

تعليلة Exists:

تستخدم تعليلة exists في لغة SQL لاختبار ما إذا كان الاستعلام الجزئي (Subquery) يعيد أي نتائج أم لا.

1- مفهوم تعليلة exists:

- تعيد القيمة True إذا كان الاستعلام الجزئي المطبق عليها غير فارغ (أي يحتوي على سطر واحد على الأقل).
- تعتبر طريقة أخرى (بديلة عن in أو التقاطع intersect) للوصول إلى نتائج معينة.

■ مثال:

إيجاد جميع المقررات التي أعطيت في كلا الفصلين "خريف 2009" و"ربيع 2010".

```

select course_id
from section as S
where semester = 'Fall' and year = 2009 and
exists (select*
        from section as T
        where semester = 'Spring' and year = 2010
        and T.course_id = S.course_id);

```

هنا يقوم الاستعلام الخارجي بالمرور على مواد خريف 2009, وبواسطة exists يتأكد: "هل توجد هذه المادة نفسها في قائمة مواد ربيع 2010؟"

2- الاستعلامات الجزئية المرتبطة (Correlated Subqueries):

لاحظ وجود ربط بين الاستعلام الداخلي والخارجي في المثال السابق من خلال الشرط $T.course_id = S.course_id$, **Correlation Name / Correlation Variable**: هو الاسم المستعار (مثل S و T) الذي نستخدمه لتمييز الجداول عند الإشارة إليها.

ملاحظات اربيسيزية:

- ماذا نكتب داخل ال select في ال exists؟
- لا يهم ما نكتبه بعد select داخل القوسين (سواء كتبنا * أو 1 أو اسم عمود معين)، لأن المهم لتعليمة exists هو وجود سطر محقق للشرط بغض النظر عن محتواه.
- الاستخدام الشائع: exists مفيدة جدا خاصة في الاستعلامات المعقدة التي تتطلب التأكد من "عدم وجود" شيء باستخدام not exists.



تعليمة Not Exists:

تستخدم تعليمة not exists للتأكد من أن الاستعلام الجزئي فارغ تماماً (لا يعيد أي أسطر).
تستخدم بشكل أساسي لحل الاستعلامات التي تحتوي على كلمة "كل"، وهي من أصعب أنواع الاستعلامات في SQL.

1. منطق "الكل" باستخدام not exists:

بدلاً من البحث عن "أخذ كل المواد"، نبحث عن "لا يوجد مادة (من المجموعة المطلوبة) لم يأخذها".

■ مثال: إيجاد الطلاب الذين أخذوا جميع المواد التي يقدمها قسم البيولوجيا.

```
select distinct S.ID, S.name
from student as S
where not exists(
(select course_id from course where dept_name = 'Biology')
except
(select T.course_id from takes as T where S.ID = T.ID));
```

2. شرح آلية التنفيذ:

العملية كعملية طرح مجموعات:

- المجموعة الأولى (X): هي كافة مواد قسم البيولوجيا.
- المجموعة الثانية (Y): هي المواد التي حضرها طالب معين (يتم فحص الطلاب واحداً تلو الآخر).
- العملية: نقوم بطرح مواد الطالب من مواد البيولوجيا ($X - Y$).
- النتيجة:

إذا كانت النتيجة خالية (Empty Set)، فهذا يعني أن الطالب قد غطى كافة مواد البيولوجيا (أي $X \subseteq Y$)، وهنا تعيد not exists القيمة True ويظهر اسم الطالب، حتى لو كان لديه مواد أخرى (Y أكبر من X).
إذا بقي ولو مادة واحدة في ناتج الطرح، فهذا يعني أن الطالب لم يأخذ تلك المادة، فتعيد not exists القيمة False ويُستبعد الطالب.

لماذا لا نستخدم = all (أو مثيلاتها some و any)؟

هذا النوع من الاستعلامات (التي تتطلب شمولية المجموعة) لا يمكن كتابته باستخدام all أو توابع التجميع البسيطة، لأننا بحاجة لمقارنة "مجموعة بمجموعة" وليس "قيمة بمجموعة".

■ أهمية ال distinct:

استخدام distinct عند عد المواد أو فحصها، لأن الطالب قد يكون قد رسب في مادة وأعادها، ونحن نهتم بمعرفة هل "أنهى" المادة وليس كم مرة حضرها.

ملاحظة هامة: أكد الدكتور على ضرورة فهم المبدأ الرياضي ($X - Y = \phi \Leftrightarrow X \subseteq Y$)
لأنه هو الأساس البرمجي الذي تعمل عليه هذه التعليمة.

تعليمة Unique:

تشبه الـ exists، وتستخدم للتأكد ما إذا كانت نتيجة الاستعلام الجزئي لا تحوي أسطراً مكررة. تعيد القيمة True عند عدم وجود أي تكرار، حتى إذا كانت النتيجة Null.

■ مثال: إيجاد المقررات التي تم تقديمها على الأكثر مرة واحدة في عام 2009.

```
select C.course_id
from course as C
where unique (select S.course_id
              from section as S
              where S.course_id = C.course_id
              and S.year = 2009);
```

في هذا المثال، يتم استبعاد أي مادة وُجدت في أكثر من سطر، وبالتالي لن يبقى إلا مواد تم إعطاؤها مرة واحدة على الأكثر.

تعليمة Not Unique:

بعكس unique، ترجع True في حال احتواء نتيجة الاستعلام الجزئي على أسطر مكررة.

العلاقات المشتقة (Derived Relations):

تستخدم هذه الطريقة عندما نريد استخدام "نتيجة استعلام" وكأنها جدول مؤقت موجود فعلياً في قاعدة البيانات، وذلك من خلال وضع استعلام select داخل مقطع الـ from.

1. استعلامات جزئية في مقطع from:

بدلاً من كتابة اسم جدول حقيقي بعد from (مثل instructor)، نفتح قوسين ونكتب استعلاماً كاملاً، الاستعلام الخارجي سيعامل هذه النتيجة كأنها جدول حقيقي ويختار منها البيانات.

■ مثال تطبيقي: إيجاد متوسط الرواتب في الأقسام التي يزيد متوسط الرواتب فيها على 42,000 دولار.

```
select dept_name, avg_salary
from (select dept_name, avg(salary) as avg_salary
      from instructor
      group by dept_name)
where avg_salary > 42000;
```



2. ملاحظة حول ال where وال having:

نحن في العادة نستخدم having لوضع شرط على التجميع (مثل المتوسط).
 • لكن هنا: بما أننا حسبنا المتوسط في "الاستعلام الداخلي" وأعطيناه اسماً (avg_salary): أصبح بالنسبة للاستعلام الخارجي مجرد "قيمة عادية" في عمود، ولذلك استطعنا استخدام where بدلاً من having.
هذه الطريقة قد تُستخدم أحياناً لتبسيط منطق الحل في المسائل المعقدة

3. تسمية العلاقات المشتقة (Aliasing):

عند استخدام استعلام في مقطع ال from فإنه من الضروري (في أغلب الأنظمة) إعطاؤه اسماً مستعاراً باستخدام as. يمكن تسمية الجدول الناتج بالكامل، كما يمكن تسمية الأعمدة الناتجة بأسماء جديدة لتسهيل استخدامها في الاستعلام الخارجي، وعند استخدام اسم مستعار في استعلام ال from يجب استخدامه ذاته في الاستعلام الخارجي.

■ مثال: طريقة أخرى للمثال السابق:

```
select dept_name, avg_salary
from (select dept_name, avg (salary)
      from instructor
      group by dept_name) as dept_avg (dept_name, avg_salary)
where avg_salary > 42000;
```

هنا قمنا بتسمية الجدول dept_avg وأعطيناه أعمدته أسماء واضحة، واستخدمناها في ال select الخارجي.

4. الفرق بينها وبين ال with:

تعليلة with (التي ستمر معنا) تشبه هذه الفكرة ولكنها تجعل الكود أنظف وأكثر تنظيماً من خلال تعريف "الجدول المؤقت" في البداية قبل الاستعلام الرئيسي.

الاستعلامات الجزئية القياسية (Scalar Subquery):

بعد أن تعلمنا وضع الاستعلام داخل ال from g where؛ نوضح الآن مكاناً ثالثاً وهو وضع الاستعلام داخل قائمة ال select مباشرة.

1. مفهوم الاستعلام القياسي:

يُسمى "قياسياً" (Scalar) لأنه يجب أن يعيد قيمة واحدة فقط (مثل رقم أو نص واحد)، ليتم عرضها كعمود بجانب بقية النتائج.

■ مثال: عرض أسماء كافة الأقسام مع "عدد المدرسين" في كل قسم.

```
select dept_name,
(select count(*)
from instructor
where department.dept_name = instructor.dept_name)
as num_instructors
from department;
```

كيف يعمل؟

يقوم الاستعلام الخارجي بالمرور على الأقسام واحداً تلو الآخر، وفي كل مرة ينفذ الاستعلام الداخلي (العد) لهذا القسم تحديداً.

ملاحظات الدكتور التقنية:

مكان الاستخدام:

نأخذ قيمة من جدول ونقوم بعملية حسابية مخصصة لها من جدول آخر في نفس السطر.

شرط القيمة الواحدة:

إذا أعاد الاستعلام الداخلي أكثر من قيمة (أكثر من سطر أو عمود)، سيعطيك النظام خطأ (Runtime Error) لأن العمود في النتيجة النهائية لا يمكن أن يتسع لأكثر من قيمة في الخلية الواحدة.

الربط (Correlation):

لاحظ وجود ربط (instructor.dept_name=department.dept_name) لضمان أن العد يتم فقط للمدرسين التابعين للقسم الحالي في الاستعلام الخارجي.

تعليلة الـ with (الجدول المؤقتة):

هي من أهم الأدوات لتبسيط الاستعلامات الطويلة والمعقدة:

المفهوم والفوائد:

- تعمل الـ with على إنشاء جدول مؤقت (Temporary View) يُستخدم فقط أثناء تنفيذ هذا الاستعلام.
- لماذا نستخدمها؟ بدلاً من كتابة استعلامات متداخلة صعبة القراءة، نقوم بتقسيم المسألة إلى أجزاء (مسائل أصغر)، نعرف كل جزء في البداية بـ with ثم نربطهم في الاستعلام النهائي.



لو أردنا الأقسام التي مجموع رواتبها أكبر من "متوسط" مجموع الرواتب لجميع الأقسام:

- نعرف جزءاً يحسب "مجموع رواتب كل قسم" ونسميه مثلاً dept_total.
- نعرف جزءاً ثانياً يحسب "متوسط هذه المجاميع".
- نكتب الاستعلام النهائي الذي يقارن بينهما.

ملاحظة: ال with لا تخزن بيانات في الهارد، بل هي "طريقة تفكير تقسيمية" (Modular) تريح المبرمج وتجعل الكود أسرع في التتبع.

- مثال: (إيجاد الأقسام ذات الميزانية الأعلى):

بدلاً من كتابة استعلام متداخل معقد، نقوم بتعريف جدول مؤقت يحتوي على "أعلى قيمة ميزانية" ثم نقارن به:

```
with max_budget (value) as
(select max(budget)
from department)
select budget
from department, max_budget
where department.budget = max_budget.value;
```

هنا قمنا بحساب ال-max مرة واحدة وخرناه في max_budget ثم استخدمناه في ال-select الرئيسية.

- مثال أكثر تعقيداً عن ال with clause:

إيجاد كافة الأقسام التي يكون "مجموع رواتب مدرسيها" أكبر من "متوسط مجموع الرواتب" لكل الأقسام

```
with dept_total (dept_name, value) as
(select dept_name, sum(salary)
from instructor
group by dept_name),
dept_total_avg(value) as
(select avg(value)
from dept_total)
select dept_name
from dept_total, dept_total_avg
where dept_total.value >= dept_total_avg.value;
```



➤ هذا الاستعلام مقسم برمجياً إلى ثلاثة أجزاء (مسائل فرعية):

الجزء الأول (dept_total):

يقوم بحساب مجموع الرواتب لكل قسم على حدة، وينتج عنه جدول مؤقت يحتوي على اسم القسم وقيمة المجموع.

الجزء الثاني (dept_total_avg):

يأخذ النتائج من الجزء الأول (المجاميع) ويحسب "المتوسط العام" لها. أي أنه يحسب رقماً واحداً يمثل متوسط ما تدفعه الأقسام كرواتب.

الاستعلام النهائي (الرئيسي):

يقوم بمقارنة مجموع رواتب كل قسم (من الجدول الأول) مع المتوسط العام (من الجدول الثاني)، ويعرض فقط أسماء الأقسام التي تزيد رواتبها عن هذا المتوسط.

Relational Database Design Normalization

تصميم قواعد المعطيات العلاقاتية

➤ ماذا نعني بتصميم قواعد المعطيات العلاقاتية؟

يقصد بعملية التصميم تحديد البنية الهيكلية للنظام، وتشمل:

- تحديد الجداول (Relations) المطلوبة وأسمائها.
- تحديد طرق الربط بين الجداول باستخدام المفاتيح الأجنبية (Foreign Keys).
- تحديد الواصفات (Attributes) أو الأعمدة التي ستوضع داخل كل جدول.
- تحديد المفاتيح الأساسية (Primary Keys) لكل جدول لتمييز السجلات.

➤ ما هي معايير التصميم الجيد؟

- تغطية كافة المتطلبات: أن يلبي التصميم جميع احتياجات المسألة أو الشركة البرمجية.
- أقل عدد من الجداول: الوصول للهدف بأبسط هيكلية ممكنة دون تعقيد غير مبرر.
- تجنب التكرار (Redundancy): وهو المعيار الأهم، لضمان أن المعلومة تُخزن في مكان واحد فقط (مصدر واحد للحقيقة)، مما يمنع حدوث "تناقضات" عند التعديل.
- تجنب مشاكل البيانات (Anomalies): التصميم الجيد يحميك من مشاكل "الحذف الخاطئ" أو "صعوبة الإضافة" أو "تكرار التعديل" في أماكن متعددة.

هل التصميم الجيد وحيد؟

الإجابة هي لا: فالتصميم الجيد ليس بالضرورة أن يكون وحيداً. قد يصل مصممان مختلفان إلى هيكليات جداول مختلفة وكلاهما يعتبر "جيداً" طالما أنهما يحققان المعايير السابقة ويصلان إلى الأشكال النظامية المطلوبة (مثل NF3) دون تكرار أو مشاكل في البيانات.

نظرية الاستنظام (Normalization)

- هي الميزان أو "الطريقة الرياضية" التي نستخدمها لتقييم جودة التصميم.
- إذا وجدنا أن التصميم سيئ، نستخدم هذه النظرية للقيام بعملية التقسيم (Decompose)، حيث نقسم الجدول الكبير السيئ إلى جداول أصغر ذات تصميم جيد تعتمد على مفهوم "الاعتمادية الوظيفية".

قائمة الأشكال النظامية (Normal Forms):

- 1st NF (الشكل النظامي الأول - First Normal Form)
- 2nd NF (الشكل النظامي الثاني - Second Normal Form)
- 3rd NF (الشكل النظامي الثالث - Third Normal Form)
- BCNF (شكل بويس - كود النظامي - Boyce -Codd Normal Form)
- NFth (الشكل النظامي الرابع)
- 5th NF (الشكل النظامي الخامس)
- Domain-key normal form

كل شكل من هذه الأشكال يغطي ما قبله بشكل متسلسل، فلا يمكن تحقيق النموذج الثاني دون تحقيق النموذج الأول، ولا تحقيق النموذج الثالث دون تحقيق الأول والثاني، وهكذا...

الشكل النظامي الأول (First Normal Form - 1NF):

يعتمد هذا المستوى بشكل كامل على مفهوم "الذرية" (Atomicity)، وهو حجر الأساس في تصميم الجداول.

1. القاعدة الأساسية:

تعتبر العلاقة (الجدول) في الشكل النظامي الأول إذا كانت كافة مجالات واصفاتها أصغرية (Atomic)، أي أن القيم في كل خلية بسيطة وغير قابلة للتقسيم.

2. الممنوعات (المجالات غير الأصغرية):

لتحقيق هذا الشكل، يجب تجنب أو معالجة ثلاثة أنواع من الوصفات:

1. الوصفات المركبة (Composite): مثل "الاسم الثلاثي": حيث يجب تقسيمه لأعمدة مستقلة (اسم، اسم أب، كنية).
2. الوصفات متعددة القيم (Multivalued): مثل "أرقام الهاتف" أو "أسماء الأولاد": حيث يمنع وضع أكثر من قيمة في خلية واحدة.
3. أرقام المحددات القابلة للتجزئة (IDs): مثل الرقم الجامعي (CS0012) الذي يشير جزء منه للقسم (CS)، فهذا يعتبر غير ذري برمجياً لأنه يمكن استنتاج معلومات من أجزائه.

نعالج هذه الأنواع أثناء التحويل من نموذج الكيانات للنموذج العلاقتي، فنقوم بتقسيم الوصفات المركبة، ووضع الوصفات متعددة القيم في جداول منفصلة:

3. مثال التحويل:

بدلاً من وضع كافة البيانات في جدول واحد بصفات معقدة، يتم تقسيمها كالتالي:

```
instructor(ID,
  first_name, middle_initial, last_name,
  street_number, street_name,
  apt_number, city, state, zip_code,
  date_of_birth)
```

```
inst_phone ( ID, phone_number)
```

instructor
ID
name
first_name
middle_initial
last_name
address
street
street_number
street_name
apt_number
city
state
zip
{ phone_number }
date_of_birth
age ()

■ جدول instructor:

يضم الوصفات بعد تقسيمها

```
(ID, first_name, middle_initial, last_name, street_number, city, state, zip_code,
date_of_birth).
```

■ جدول inst_phone:

يضم رقم الهاتف لكونه واصفة متعددة القيم.

ملاحظة: نموذج العلاقات (Relational Model) يفرض الـ NF1 آلياً لأن أنظمة قواعد البيانات لا تدعم "المجموعات" كنوع بيانات داخل الخلية الواحدة.

■ مثال: التحويل إلى الشكل النظامي الأول (NF1):

يوضح هذا المثال كيف تُعالج الجداول التي تحتوي على واصفات متعددة القيم Multivalued Attributes. لتصبح متوافقة مع شروط الـ NF1.

DEPARTMENT			
DNAME	DNUMBER	DMGRSSN	DLOCATIONS
Research	5	333445555	{Bellaire, Sugarland, Houston}
Administration	4	987654321	{Stafford}
Headquarters	1	888665555	{Houston}

❗ المشكلة :

لدينا جدول الأقسام (DEPARTMENT) وفيه واصفة هي "أماكن القسم" (DLOCATIONS). نلاحظ في البيانات أن قسم الأبحاث (Research) موجود في ثلاثة أماكن معاً {Bellaire, Sugarland, Houston} وضعت كلها في خلية واحدة.

برمجياً: هذا التصميم يخرق قاعدة الذرية (Atomicity) لأن الخلية الواحدة تحتوي على "مجموعة" قيم وليس قيمة واحدة بسيطة.

DEPARTMENT			
DNAME	DNUMBER	DMGRSSN	DLOCATION
Research	5	333445555	Bellaire
Research	5	333445555	Sugarland
Research	5	333445555	Houston
Administration	4	987654321	Stafford
Headquarters	1	888665555	Houston

❗ الحل :

لتحقيق الـ NF1، نقوم بـ "فرط" أو تقسيم هذه المجموعة، بحيث ننشئ سطراً مستقلاً لكل مكان.

النتيجة: بدلاً من سطر واحد لقسم الأبحاث، أصبح لدينا 3 أسطر، كل سطر يحتوي على مكان واحد فقط (DLOCATION).

ضريبة التحويل (التكرار):

■ بمجرد ما حولنا الجدول للـ NF1، ظهرت مشكلة جديدة وهي التكرار (Redundancy): حيث تكرر اسم القسم ورقم المدير في كل سطر.

- موقف الـ NF1 من التكرار: الشكل النظامي الأول لا يهتم بالتكرار، هدفه الوحيد والأساسي هو أن تكون البيانات "ذرية" فقط. معالجة التكرار ستبدأ من الشكل النظامي الثاني (NF2) وما بعد.
- تغيير المفتاح الرئيسي: في الجدول القديم كان رقم القسم (DNUMBER) يكفي ليكون مفتاحاً رئيسياً، أما بعد التقسيم لم يعد يكفي بسبب تكراره، وأصبحنا بحاجة لمفتاح مركب يتكون من (رقم القسم + المكان) لتمييز الأسطر.
- سطر واحد لقسم الأبحاث، أصبح لدينا 3 أسطر، كل سطر يحتوي على مكان واحد فقط (DLOCATION).

ملاحظة: لا يوجد نوع بيانات يعبر عن المجموعات، فالجدول السابق افتراضي لشرح فكرة التحويل.

مشاكل التصميم السيئ (Anomalies):

- يحوي الجدول التالي: رقم الموظف، ورقم المشروع، واسم الموظف، واسم المشروع، وعدد ساعات عمل الموظف في المشروع:
EMP_PROJ (Emp#, Proj#, Ename, Pname, No_hours)

1. مشكلة التعديل (Update Anomaly):

تحدث عندما نضطر لتعديل معلومة واحدة في أماكن كثيرة جداً.

- مثال: إذا أردنا تغيير اسم المشروع "Billing" إلى "Customer-Accounting"، وكان يعمل في هذا المشروع 100 موظف، سنضطر لتعديل الاسم في 100 سطر.

• الخطر: إذا نسينا سطرًا واحدًا فقط، سيصبح لدينا "تناقض في البيانات"؛ حيث يظهر نفس المشروع باسمين مختلفين في نفس الجدول.

2. مشكلة الإضافة (Insert Anomaly):

تحدث عندما يعجز النظام عن إضافة معلومة جديدة لأنها تفتقر لمعلومة أخرى مرتبطة بها في نفس الجدول.

- مثال: لا يمكننا إضافة "مشروع جديد" إلى قاعدة البيانات إلا إذا قمنا بتعيين موظف واحد على الأقل فيه فوراً.

• العكس صحيح: قد لا نستطيع إضافة بيانات "موظف جديد" إذا لم يكن قد تم فرزها لمشروع معين بعد.

3. مشكلة الحذف (Delete Anomaly):

وهي الأخطر، حيث يؤدي حذف سجل معين إلى ضياع معلومات هامة أخرى بالخطأ.

- مثال: إذا حذفنا "مشروعاً" معيناً، سيقوم النظام آلياً بحذف كافة بيانات "الموظفين" الذين يعملون في هذا المشروع، وإذا لم يكونوا يعملون على مشروع آخر، أو لم تكن بياناتهم مخزنة في جدول آخر، فسنفقدّها للأبد.

ملاحظة: إذا كان هناك موظف وحيد يعمل في مشروع، وحذفنا هذا الموظف، سيختفي المشروع تماماً من النظام وكأنه لم يكن.

نظرية الاستنظام (Normalization Theory)

هذه النظرية هي "المسطرة الرياضية" التي نقيس بها جودة تصميم الجداول. إذا وجدنا تصميمًا سيئاً (يحتوي على تكرار)، نستخدم هذه النظرية أيضاً للقيام بعملية التقسيم (Decompose) إلى جداول أصغر ذات تصميم جيد بدون ضياع للمعلومات (Lossless Decomposition).

1. الأداة الأساسية: الاعتمادية الوظيفية (Functional Dependency):

تعتمد النظرية بالكامل على مفهوم الاعتمادية الوظيفية (سهم يشير من A -المعتمد عليه- إلى B -المعتمد-).

• معناها: إذا عرفت قيمة A، فأنا حتماً أعرف قيمة B.

القاعدة الذهبية: في التصميم الجيد، يجب أن تكون كافة الاعتماديات مبنية على المفتاح فقط

2. مثال تطبيقي (جدول المدرسين والأقسام المدمج):

ID	name	salary	dept_name	building	budget
22222	Einstein	95000	Physics	Watson	70000
12121	Wu	90000	Finance	Painter	120000
32343	El Said	60000	History	Painter	50000
45565	Katz	75000	Comp. Sci.	Taylor	100000
98345	Kim	80000	Elec. Eng.	Taylor	85000
76766	Crick	72000	Biology	Watson	90000
10101	Srinivasan	65000	Comp. Sci.	Taylor	100000
58583	Califieri	62000	History	Painter	50000
83821	Brandt	92000	Comp. Sci.	Taylor	100000
15151	Mozart	40000	Music	Packard	80000
33456	Gold	87000	Physics	Watson	70000
76543	Singh	80000	Finance	Painter	120000

تحليل الاعتماديات في هذا الجدول:

1. اعتمادية صحيحة:

ID -> name, salary, dept_name

(لأن الاسم والراتب يعتمدان على رقم المدرس "المفتاح").

2. اعتمادية "سيئة" (سبب التكرار):

dept_name -> building, budget.

لماذا هي سيئة؟

لأن المبنى والميزانية يعتمدان على "اسم القسم" وليس على "رقم المدرس".

النتيجة:

طالما أن dept_name ليس مفتاحاً لهذا الجدول المدمج، فسيؤدي ذلك لتكرار بيانات المبنى والميزانية مع كل مدرس ينتمي لنفس القسم.

3. الحل عن طريق التقسيم (Decomposition):

لحل هذه المشكلة، نقوم بتقسيم الجدول الكبير إلى جدولين بحسب الاعتماديات الوظيفية التي استنتجناها:

1. جدول (1): يضم المدرسين (ID, name, salary, dept_name); هنا ID هو المفتاح.
2. جدول (2): يضم الأقسام (dept_name, building, budget); هنا dept_name هو المفتاح.

نستنتج هذه الاعتماديات بمجرد النظر للبيانات: فكلما رأينا "Physics" نجد بجوارها دائماً "Watson" و"70000"، وهذا يعني وجود اعتمادية وظيفية.

الهدف من التقسيم:

- بهذا التقسيم حققنا مبدأ "مصدر واحد للحقيقة"؛ فإذا تغيرت ميزانية قسم الفيزياء K سنعدلها في سطر واحد فقط في جدول الأقسام، ولن نضطر لتعديلها في كل أسطر المدرسين.
 - الربط:
- بعد التقسيم نربط الجدولين عن طريق المفتاح الأجنبي (dept_name) لضمان عدم ضياع العلاقة بين المدرس وقسمه.

الشكل النظامي الثاني والثالث (Normalization into 2nd NF and 3rd NF):

"كل شيء يجب أن يعتمد على المفتاح، المفتاح كاملاً، ولا شيء غير المفتاح"

1. الشكل النظامي الثاني (2NF): "المفتاح كاملاً"

- يهدف هذا الشكل إلى التخلص من "الاعتمادية الجزئية"؛ أي لا يجوز لأي معلومة أن تعتمد على "جزء" من المفتاح إذا كان المفتاح مركباً.

■ مثال : جدول EMP_PROJ

لدينا جدول يضم (رقم الموظف SSN, رقم المشروع PNUMBER, عدد الساعات, اسم الموظف, اسم المشروع, مكان المشروع):



■ المفتاح الرئيسي هنا مركب: (رقم الموظف + رقم المشروع).

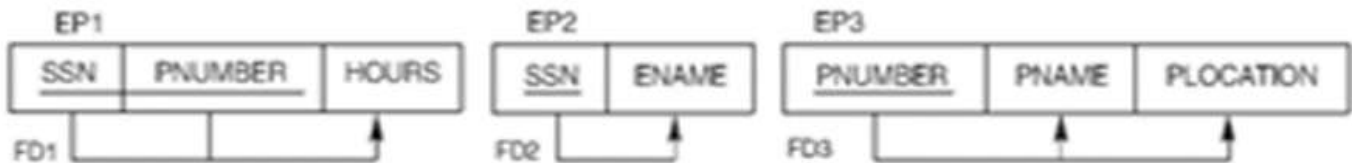
تحليل المشاكل (الاعتماديات الجزئية):

اعتمادية صحيحة: "عدد الساعات" تعتمد على الموظف والمشروع معاً.

■ اعتمادية جزئية (1): "اسم الموظف" يعتمد على SSN فقط (جزء من المفتاح).

■ اعتمادية جزئية (2): "اسم المشروع ومكانه" يعتمدان على PNUMBER فقط (جزء من المفتاح).

الحل بالتقسيم (Decomposition) لثلاثة جداول:



■ EP1: يضم (SSN, PNUMBER, HOURS) لربط الموظف بالمشروع.

■ EP2: يضم (SSN, ENAME) لبيانات الموظفين.

■ EP3: يضم (PNUMBER, PNAME, PLOCATION) لبيانات المشاريع.



Let it Fall,
Let it Flow,
Let it Teach you
How To Grow..

2. الشكل النظامي الثالث (NF3): "ولا شيء غير المفتاح"

يهدف هذا الشكل إلى التخلص من "الاعتمادية المتعدية": أي لا يجوز لمعلومة (ليست مفتاحاً) أن تعتمد على معلومة أخرى (ليست مفتاحاً أيضاً).

■ المثال التطبيقي: جدول EMP_DEPT

لدينا جدول يضم (اسم الموظف، رقم الموظف SSN، تاريخ الميلاد، العنوان، رقم القسم، اسم القسم، رقم مدير القسم).



• المفتاح الرئيسي: رقم الموظف (SSN).

تحليل المشكلة (الاعتمادية المتعدية):

- رقم الموظف (SSN) يحدد "رقم القسم" (DNUMBER).
- ورقم القسم (DNUMBER) يحدد "اسم القسم ومديره".

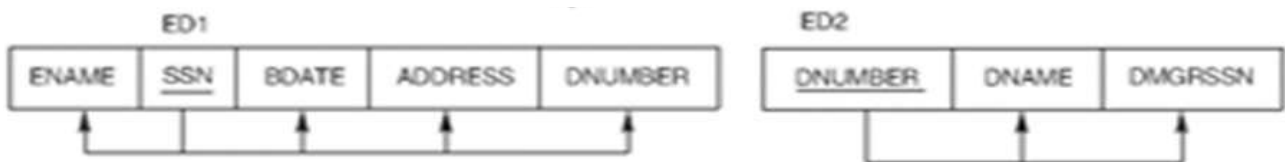
• النتيجة:

اسم القسم يعتمد على رقم الموظف بشكل "متعد" عبر رقم القسم، وهذا يسبب تكراراً لبيانات القسم مع كل موظف.

الحل بالتقسيم (Decompose) لجدولين:

■ ED1: يضم بيانات الموظف مع رقم قسمه (ENAME, SSN, BDATE, ADDRESS, DNUMBER).

■ ED2: يضم بيانات القسم المستقلة (DNUMBER, DNAME, DMGRSSN).

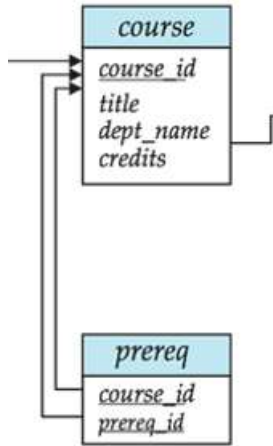


- الفرق الجوهرية: في الـ NF2 المشكلة تكون في "المفتاح" (الاعتماد على جزء منه)، بينما في الـ NF3 المشكلة تكون في "الواصفات العادية" (اعتماد واطقة على واطقة أخرى).
- التفكير المنطقي: إذا صممت الـ ERD بشكل صحيح منذ البداية وفصلت الكيانات (موظف، مشروع، قسم)، فستصل لهذه الأشكال النظامية تلقائياً دون الحاجة لهذه التعقيدات الرياضية.
- الأتمتة: هذه النظرية تسمح "بأتمتة" التصميم؛ حيث يمكنك إعطاء البرنامج كافة البيانات في جدول واحد مع "الاعتماديات"، وهو سيقوم آلياً بتقسيمها لأفضل شكل ممكن.

تجاوز الاستنظام لرفع الأداء (Denormalization)

في بعض الحالات نضطر لكسر قواعد الاستنظام (التي تعبنا في تحقيقها) والقبول بوجود تكرار لسبب وجيد وهو سرعة القراءة (Speed).

1- المشكلة: تكلفة الربط (Join Cost):



- في التصميم المستنظم، لدينا جدول للمواد وجدول للمتطلبات السابقة (prereq) يحتوي فقط على أرقام المواد.
- لعرض "أسماء" المواد ومتطلباتها، يضطر النظام للقيام بعملية Join، وهي عملية مكلفة جداً تقنياً وتستغرق وقتاً طويلاً ($N \times N$).

2- الحل: دمج البيانات (Denormalization):

- نقوم بإضافة "اسم المادة" و"اسم المتطلب" مباشرة داخل جدول المتطلبات:
- ```
prereq (course_id, course_title, prereq_id, prereq_title)
```
- النتيجة: لم نعد بحاجة للJoin، وأصبحت القراءة سريعة جداً (Faster Lookup).

نلاحظ أن المفتاح الأساسي في جدول المتطلبات هو `course_id` و `prereq_id` معاً، وبعد الدمج أصبح `course_title` يعتمد على `course_id` (جزء من المفتاح الأساسي)، و `prereq_title` يعتمد على `prereq_id` (جزء من المفتاح الأساسي)، أي لم يعد الشكل النظامي الثاني محققاً (كل شيء يجب أن يعتمد على المفتاح كاملاً)، ولكن في المقابل استطعنا تحسين الأداء.

### ضريبة كسر القواعد:

1. تكرار واستهلاك مساحة: البيانات أصبحت مخزنة في أكثر من مكان.
2. بطء التعديل: إذا تغير اسم مادة واحدة، سنضطر لتعديلها في كل الجداول التي كررنا فيها هذا الاسم، وإلا سيحدث تناقض.
3. تعقيد الكود: يزداد الجهد البرمجي لضمان تحديث كافة النسخ المكررة، ويزداد احتمالية ورود أخطاء في الكود.

قاعدة عامة: نلجأ لهذا الحل لأن عمليات "القراءة" في أغلب الأنظمة أكثر بكثير من عمليات "التعديل".

## ملاحظات تصميمية:

هناك تصاميم قد تحقق الأشكال النظامية (مثل BCNF) ولكنها تعتبر تصاميم سيئة جداً ويجب تجنبها، وغالباً ما يقع فيها من اعتادوا على العمل على جداول "إكسل" (Excel Mindset).

■ مثال: الأرباح السنوية للشركات:

الهدف هو تخزين أرباح كل شركة في كل سنة.

### التصاميم السيئة (التي يجب تجنبها):

- إنشاء جدول لكل سنة: (أرباح\_2025، أرباح\_2026 ..)، هذا خطأ لأنه يتطلب إنشاء جدول جديد يدوياً كل سنة، ويجعل الاستعلام عبر السنوات مستحيلاً.
- إضافة عمود لكل سنة: (الشركة، ربح\_2025، ربح\_2026 ..): هذا خطأ أيضاً لأنه يتطلب تغيير بنية الجدول (Schema) بإضافة عمود جديد كل سنة، وهو تصميم غير مرن.

### التصميم الصحيح (المرن):

أن يكون لدينا جدول واحد ثابت يحتوي على ثلاثة أعمدة فقط: (رقم الشركة، السنة، المبلغ). بهذا الشكل، لإضافة سنة جديدة، نضيف "سطراً" جديداً فقط ولا نغير بنية قاعدة البيانات.



بعض الدروب نود أن لا تنتهي .. والسير أحياناً ألد من الوصول

إلى هنا نكون قد أنهينا رحلة مقرر قواعد المعطيات لهذا الفصل  
وضعنا فيه جهودنا وطاقاتنا راجين من الله أن نقدم لكم المعلومة بشكل كامل  
لا تنسونا من صالح دعائكم

- فريق الداتا ♥